

Prática II: Gerência de Memória

Roteiro de Laboratório: Hardware e Uso de Memória

Curso de Engenharia de Telecomunicações
IFSC Campus São José
Sistemas Operacionais (Prof. Rogério Pereira Junior)

19 de junho de 2026

Introdução

Neste laboratório investiguei o subsistema de memória do Linux. Os tópicos cobertos foram o tamanho de página e os limites do sistema, a alocação dinâmica com *Lazy Allocation*, a diferença entre memória virtual (VSZ) e física (RSS), a inspeção da tabela de páginas com o `pmap`, as estatísticas globais de paginação e de *swap*, o mapeamento de arquivos com `mmap` e os segmentos de memória de um processo. Todos os valores foram coletados em um ambiente Linux real (kernel sobre WSL2, com página de 4 KB).

Sumário

1 Roteiro Prático II: Hardware de Memória	1
1.1 Tamanho da Página e Limites do Sistema	1
1.2 Alocação Dinâmica e Páginas Virtuais vs. Físicas	2
1.3 Compilação e Monitoramento (<code>ps</code> e <code>/proc</code>)	2
1.4 Inspeccionando a Tabela de Páginas com <code>pmap</code>	3
1.5 Páginas Globais e Atividade de <i>Swap</i> (<code>/proc/meminfo</code> e <code>vmstat</code>)	4
1.6 Mapeamento de Arquivos em Páginas (<code>mmap</code>)	4
1.7 Monitoramento de paginação em tempo real (<code>sar -B</code>)	6
2 Roteiro Prático II: Uso de Memória	6
2.1 Segmentos de um processo (<code>hello.c</code> e <code>pmap</code>)	6
2.2 Alocação estática, automática e dinâmica (<code>alocacao.c</code>)	6
2.3 Recursão e a Pilha (<code>fatorial.c</code>)	7
3 Considerações Finais: a função da MMU	8

1 Roteiro Prático II: Hardware de Memória

1.1 Tamanho da Página e Limites do Sistema

Comandos executados:

```
$ getconf PAGESIZE
4096
$ sysctl kernel.shmall
kernel.shmall = 18446744073692774399
```

Pergunta 1: tamanho de página. O valor retornado foi 4096 bytes, que convertido equivale a:

$$\frac{4096\text{bytes}}{1024} = 4KB.$$

Esse número é o tamanho do bloco fixo (página) em que o subsistema de memória do Linux divide tanto a memória virtual de cada processo quanto a memória física (a RAM, dividida em quadros, ou *frames*). Toda alocação e todo mapeamento de endereços virtuais para físicos é feito em múltiplos de 4 KB, que é também a granularidade mínima de uma falta de página (*page fault*). A página é a unidade básica de gerência da MMU e da tabela de páginas.

Pergunta 2: limite de memória compartilhada. Multiplicando `kernel.shmall` pelo tamanho da página:

$$18.446.744.073.692.774.399 \times 4096\text{bytes} \approx 7,03 \times 10^{22}\text{bytes}.$$

Em GB isso corresponde a aproximadamente 7×10^{13} GB. Na prática, esse é o valor padrão do kernel (próximo do limite teórico de um inteiro de 64 bits), o que significa que o `shmall` não impõe limite prático para alocação de memória compartilhada neste sistema. O fator limitante real passa a ser a RAM física e o espaço de *swap* disponíveis. Em sistemas onde o administrador define um `shmall` menor, esse produto fornece o teto real de memória compartilhada.

1.2 Alocação Dinâmica e Páginas Virtuais vs. Físicas

O programa `paginacao.c` aloca 16 MB com `malloc` e só escreve neles em um segundo momento, o que permite observar o efeito da *Lazy Allocation*.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5
6 #define DEZESSEIS_MEGA (16 * 1024 * 1024) // 16 MB
7
8 int main() {
9     int pid = getpid();
10    printf("=== PROCESSO INICIADO (PID: %d) ===\n", pid);
11    printf("Passo A: Antes de alocar memoria. Verifique o consumo.\n");
12    printf("Pressione ENTER para alocar 16 MB (mas sem usar)...");
13    getchar();
14
15    // Alocando 16 Megabytes de memoria virtual
16    char *memoria = (char *)malloc(DEZESSEIS_MEGA);
17    if (memoria == NULL) {
18        perror("Erro ao alocar");
19        return 1;
20    }
21
22    printf("\nPasso B: 16 MB alocados virtualmente.\n");
23    printf("Pressione ENTER para ESCREVER nas paginas de memoria...");
24    getchar();
25
26    // Escrevendo em cada pagina (pulando de 4096 em 4096 bytes)
27    // Isso força o Kernel a alocar os quadros fisicos na RAM
28    for (int i = 0; i < DEZESSEIS_MEGA; i += 4096) {
29        memoria[i] = 'X';
30    }
31
32    printf("\nPasso C: Dados escritos na memoria (Paginas fisicas associadas).\n");
33    printf("\n");
```

```

33     printf("Pressione ENTER para encerrar o programa...");
34     getchar();
35
36     free(memoria);
37     return 0;
38 }

```

Listing 1: paginacao.c

1.3 Compilação e Monitoramento (ps e /proc)

```

$ gcc paginacao.c -o paginacao
$ ./paginacao # Terminal 1
# Terminal 2:
$ ps -o pid,vsz,rss,comm -p [PID]
$ cat /proc/[PID]/status | grep -E "VmSize|VmRSS"

```

Valores coletados em cada etapa, com o PID monitorado durante a execução real:

Momento do Programa	VSZ / VmSize (KB)	RSS / VmRSS (KB)
Passo A (Início)	2.684	1.408
Passo B (malloc feito)	19.072	1.408
Passo C (Escrita feita)	19.072	17.792

Tabela 1: Memória virtual (VSZ) e física residente (RSS) por etapa.

Pergunta: o que ocorreu no Passo B (malloc)? A memória virtual (VSZ) cresceu de 2.684 KB para 19.072 KB, um aumento de cerca de 16.384 KB (= 16 MB), que corresponde à região reservada pelo `malloc`. Já a memória física (RSS) permaneceu inalterada em 1.408 KB. O motivo é a *Lazy Allocation* (alocação preguiçosa): o `malloc` apenas reserva um intervalo de endereços no espaço virtual do processo, ajustando a tabela de páginas com entradas marcadas, mas o kernel não associa nenhum quadro físico de RAM enquanto aquelas páginas não forem efetivamente acessadas. Como ainda não houve escrita, nenhum *page fault* ocorreu e a RAM não foi consumida.

Pergunta: o que ocorreu no Passo C (escrita) e por que pular de 4096 em 4096? Depois do laço `for`, o RSS saltou de 1.408 KB para 17.792 KB (cerca de +16 MB), mostrando que agora os 16 MB virtuais passaram a ocupar quadros físicos reais na RAM. Isso acontece porque a primeira escrita em cada página dispara uma falta de página (*page fault*); o kernel então aloca um quadro físico livre, atualiza a tabela de páginas mapeando o endereço virtual ao físico e só então a escrita se conclui.

O código pula de 4096 em 4096 bytes porque esse é o tamanho da página. Basta tocar um único byte de cada página para forçar a alocação do quadro inteiro de 4 KB; escrever em todos os bytes seria desperdício de tempo, pois não traz nenhuma página física adicional. Com o passo de 4096 garantimos que todas as páginas dos 16 MB sejam materializadas na RAM, percorrendo o número mínimo de posições ($16\text{ MB}/4\text{ KB} = 4096$ páginas).

1.4 Inspeccionando a Tabela de Páginas com pmap

Com o programa parado no Passo C, executei `pmap -x [PID]`. A linha correspondente à alocação de 16 MB:

Address	Kbytes	RSS	Dirty	Mode	Mapping
00007fcf4a1ff000	16388	16384	16384	rw---	[anon]
total kB	19072	17936	16488		

Coluna	Valor (KB)
Kbytes (tamanho virtual da região)	16.388
RSS (páginas residentes em RAM)	16.384
Dirty (páginas sujas)	16.384

Tabela 2: Região anônima de 16 MB observada no `pmap -x`.

Pergunta: o que a coluna Dirty indica? A coluna Dirty (página suja) indica a quantidade de memória cujas páginas foram modificadas (escritas) após o mapeamento e cujo conteúdo na RAM difere da fonte em disco, ou que nunca teve respaldo em disco, como é o caso da memória anônima. Para o Sistema Operacional, uma página suja é uma página que não pode ser simplesmente descartada: antes de reaproveitar seu quadro físico, o kernel precisa primeiro gravá-la, seja no *swap* (no caso de memória anônima) seja de volta no arquivo (no caso de páginas mapeadas em arquivo). Uma página limpa, idêntica ao disco, pode ser descartada na hora, pois é possível relê-la depois. Como escrevemos 'X' em todas as 4096 páginas, todas as 16.384 KB aparecem como sujas.

1.5 Páginas Globais e Atividade de *Swap* (/proc/meminfo e vmstat)

```
$ cat /proc/meminfo | grep -E "Cached|Buffers"
Buffers:          87204 kB
Cached:          1933848 kB
SwapCached:        0 kB

$ vmstat 2
procs  -----memory-----  ---swap--  -----io-----  -system--  -----cpu-----
 r   b    swpd     free   buff   cache    si   so    bi    bo    in   cs  us  sy  id  wa  st
 0   0        0 12326784  87344 2028772    0   0  1465 1849 2146    2   1   1  98   0   0
 0   0        0 12326052  87344 2028780    0   0    0    0  726 1251   0   0 100   0   0
    0
```

Pergunta: o que indicam as colunas si (swap in) e so (swap out)? Sob o cabeçalho `swap` do `vmstat`:

- **si (swap in):** quantidade de memória, em KB/s, lida do disco de *swap* de volta para a RAM. Indica que páginas que haviam sido enviadas ao *swap* estão sendo trazidas de novo porque algum processo precisou delas.
- **so (swap out):** quantidade de memória, em KB/s, escrita da RAM para o disco de *swap*. Ocorre quando há pressão de memória e o kernel precisa liberar quadros físicos, movendo páginas pouco usadas para o *swap*.

Valores altos e contínuos nessas colunas indicam *thrashing*, ou seja, um sistema com falta de RAM e desempenho degradado. Nos testes, ambos ficaram em 0, já que o sistema tinha RAM livre de sobra (cerca de 11 GiB).

Pergunta: o que são *HugePages*?

```
$ cat /proc/meminfo | grep -i huge
AnonHugePages:      0 kB
HugePages_Total:    0
HugePages_Free:      0
```

Hugepagesize:	2048 kB
---------------	---------

HugePages são páginas de memória de tamanho muito maior do que a página padrão de 4 KB, tipicamente 2 MB (visível em `Hugepagesize: 2048 kB`) ou 1 GB em arquiteturas x86-64. O objetivo é reduzir a sobrecarga de gerência de memória: com páginas maiores, são necessárias muito menos entradas na tabela de páginas para cobrir a mesma quantidade de RAM, o que diminui o número de *misses* na TLB (*Translation Lookaside Buffer*, a cache de traduções da MMU) e acelera o acesso à memória. São úteis para aplicações que manipulam grandes volumes contíguos de memória, como bancos de dados (Oracle, PostgreSQL) e máquinas virtuais. Os campos `HugePages_Total` e `HugePages_Free` mostram quantas estão pré-reservadas (zero neste sistema), e `AnonHugePages` reflete o uso de *Transparent Huge Pages* (THP), em que o kernel promove páginas automaticamente.

1.6 Mapeamento de Arquivos em Páginas (mmap)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <sys/stat.h>
6 #include <unistd.h>
7
8 int main() {
9     int fd = open("texto.txt", O_RDWR);
10    if (fd < 0) {
11        perror("Erro ao abrir arquivo");
12        return 1;
13    }
14
15    struct stat sb;
16    fstat(fd, &sb);
17
18    // Mapeando o arquivo diretamente em uma pagina de memoria virtual
19    char *arquivo_em_memoria = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE,
20    MAP_SHARED, fd, 0);
21    if (arquivo_em_memoria == MAP_FAILED) {
22        perror("Erro no mmap");
23        return 1;
24    }
25
26    // O arquivo agora pode ser lido como se fosse um vetor na memoria
27    printf("Conteudo lido da pagina de memoria: %s", arquivo_em_memoria);
28
29    // Modificando a memoria diretamente. A MMU e o Kernel sincronizarao com o
30    disco.
31    arquivo_em_memoria[0] = '0';
32
33    printf("Modificacao feita na memoria. Pressione ENTER para finalizar...");
34    getchar();
35
36    munmap(arquivo_em_memoria, sb.st_size);
37    close(fd);
38    return 0;
39 }
```

Listing 2: mapeado.c

```
$ echo "Arquitetura de Computadores e Sistemas Operacionais" > texto.txt
$ gcc mapeado.c -o mapeado
$ ./mapeado
```

```
Conteúdo lido da pagina de memoria: Arquitetura de Computadores e Sistemas
Operacionais
$ cat texto.txt
Orquitetura de Computadores e Sistemas Operacionais
```

Pergunta: o que mudou em texto.txt e por quê? O primeiro caractere mudou de 'A' para 'O': o arquivo passou de "Arquitetura..." para "Orquitetura...". A alteração veio da única linha `arquivo_em_memoria[0] = 'O';`, que é uma escrita em uma posição de memória, sem nenhuma chamada a `write` ou `fprintf`.

Isso é possível por causa do `mmap` com a *flag* `MAP_SHARED`, que mapeia o conteúdo do arquivo em disco diretamente no espaço de endereçamento virtual do processo. As páginas virtuais retornadas pelo `mmap` ficam associadas ao arquivo. Quando o processo escreve em `arquivo_em_memoria[0]`, a MMU traduz o endereço virtual para o físico e a página correspondente é marcada como suja. O kernel, por meio do *page cache* e do *writeback*, sincroniza automaticamente as páginas sujas de volta ao arquivo em disco, seja no `munmap`, seja em um *flush* periódico, seja sob pressão de memória. Dessa forma, a paginação transforma operações de E/S de arquivo em simples acessos à memória, sem precisar de chamadas explícitas de escrita.

1.7 Monitoramento de paginação em tempo real (`sar -B`)

O comando `sar -B 1 10` (do pacote `sysstat`) coleta estatísticas de paginação a cada 1 segundo, 10 vezes. As principais colunas e seus significados:

- `pgpgin/s`, `pgpgout/s`: KB/s lidos do disco para a memória e gravados da memória para o disco, respectivamente.
- `fault/s`: número total de faltas de página por segundo (*minor* mais *major*).
- `majflt/s`: faltas de página maiores por segundo, aquelas que exigem acesso ao disco (a página precisa ser carregada de um arquivo ou do *swap*) e que são as mais custosas.
- `pgfree/s`: páginas liberadas pelo sistema por segundo.
- `pgscan/s`, `pgscand/s`: páginas varridas pelo *kswapd* (em segundo plano) e diretamente, em busca de páginas para recuperar.
- `pgsteal/s`: páginas recuperadas da *page cache* por segundo.
- `%vmeff`: eficiência da recuperação de páginas (`pgsteal/pgscan`); valores próximos de 100% indicam recuperação eficiente.

Observação: neste ambiente o pacote `sysstat` não estava instalado (`sudo apt install sysstat`), então a descrição acima cobre o significado de cada linha.

2 Roteiro Prático II: Uso de Memória

2.1 Segmentos de um processo (hello.c e pmap)

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int global_init = 42;    // DATA
4 int global_uninit;      // BSS
5 int main() {
6     printf("Hello World!\n");
7     sleep(600);
8 }
```

Listing 3: hello.c

A variável `global_init = 42` fica no segmento DATA (dados inicializados) e `global_uninit` no segmento BSS (dados não inicializados, zerados na carga). A saída do `pmap` mostra os segmentos do processo:

```

$ ./hello & ; pmap <PID>
4646:    ./hello
000064557c8dc000      4K r---- hello      <- .text/rodata (codigo, R0)
000064557c8dd000      4K r-x-- hello      <- codigo executavel
000064557c8de000      4K r---- hello
000064557c8df000      4K r---- hello      <- DATA / BSS
000064557c8e0000      4K rw--- hello      <- dados grav veis
00006455a39f7000    132K rw--- [ anon ]      <- HEAP
000074ca99e00000    ... libc.so.6          <- biblioteca C compartilhada
000074ca9a1d1000    ... ld-linux-x86-64.so.2 <- linker din mico
00007ffeabcd64000    136K rw--- [ stack ]      <- STACK (pilha)
total                2684K

```

Dá para ver a divisão em segmentos: código (permissões **r-x**, somente leitura e execução), dados (**rw-**), heap (**[anon]**), pilha (**[stack]**) e as bibliotecas compartilhadas (**libc** e **ld-linux**). Todas as regiões são múltiplos de 4 KB, o tamanho da página.

2.2 Alocação estática, automática e dinâmica (alocacao.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 int global = 10; // Estatica
4 void funcao_auto(int n) {
5     int local = n; // Automatica
6     printf("Local = %d\n", local);
7 }
8 int main() {
9     funcao_auto(5);
10    int *ptr = malloc(sizeof(int)); // Dinamica
11    *ptr = 99;
12    printf("Dinamica = %d\n", *ptr);
13    free(ptr);
14    return 0;
15 }

```

Listing 4: alocacao.c

```

$ ./alocacao
Local = 5
Dinamica = 99

```

Os três tipos de alocação:

- Estática (**int global = 10**): vive no segmento DATA e existe durante toda a execução do programa.
- Automática (**int local = n**): vive na pilha (STACK), criada na entrada da função e destruída no seu retorno.
- Dinâmica (**malloc**): vive no heap e é controlada manualmente pelo programador, com **malloc** e **free**.

2.3 Recursão e a Pilha (fatorial.c)

```

1 #include <stdio.h>
2 long fatorial(int n) {
3     long parcial;
4     printf("inicio: n = %d\n", n);
5     if (n < 2)
6         parcial = 1;
7     else
8         parcial = n * fatorial(n - 1);

```

```
9   printf("final : n = %d, parcial = %ld\n", n, parcial);
10  return parcial;
11 }
12 int main() {
13     printf("Fatorial(4) = %ld\n", fatorial(4));
14     return 0;
15 }
```

Listing 5: fatorial.c

```
$ ./fatorial
inicio: n = 4
inicio: n = 3
inicio: n = 2
inicio: n = 1
final : n = 1, parcial = 1
final : n = 2, parcial = 2
final : n = 3, parcial = 6
final : n = 4, parcial = 24
Fatorial(4) = 24
```

A saída mostra o comportamento de pilha (LIFO). Todas as chamadas "inicio" aparecem primeiro, na descida da recursão ($n = 4, 3, 2, 1$): cada chamada de `fatorial` empilha um novo *stack frame* com a sua própria variável local `parcial` e o parâmetro `n`, como pratos empilhados. Ao chegar no caso base ($n < 2$), os retornos acontecem na ordem inversa ($n = 1, 2, 3, 4$), desempilhando cada quadro e calculando o produto parcial. Cada nível mantém a sua cópia das variáveis locais, o que mostra que a pilha isola o contexto de cada chamada.

3 Considerações Finais: a função da MMU

A MMU (Memory Management Unit) é o componente de hardware, em geral integrado ao processador, que traduz em tempo de execução os endereços virtuais usados pelos programas em endereços físicos reais da RAM. Toda referência de memória feita por um processo passa pela MMU, que consulta a tabela de páginas mantida pelo Sistema Operacional para mapear a página virtual ao quadro físico correspondente, usando a cache TLB para acelerar as traduções recentes.

Com base nos experimentos, as funções da MMU que observei foram:

- Tradução de endereços virtuais para físicos, na granularidade de página (4 KB), de forma transparente para o programa.
- Suporte à *Lazy Allocation*: ao detectar acesso a uma página virtual ainda sem quadro físico, a MMU gera uma falta de página (*page fault*) e interrompe a CPU para que o kernel aloque o quadro. Foi exatamente o que vimos entre os Passos B e C.
- Proteção e isolamento de memória: a MMU verifica as permissões de cada página (leitura, escrita e execução) e impede que um processo acesse a memória de outro.
- Marcação de páginas como sujas ou acessadas, dando ao SO a informação necessária para decidir o que enviar ao *swap* e o que sincronizar com o disco, como no `mmap`.

A MMU é, portanto, a peça que viabiliza a memória virtual: ela separa o espaço de endereçamento lógico de cada processo da memória física e permite paginação, proteção, compartilhamento de memória e uso de *swap*, tudo de forma transparente para as aplicações.